

Hintro Backend/Fullstack Engineering Internship Assignment

Introduction

At Hintro, we build AI-powered meeting intelligence products that help users capture insights, action items, decisions, and follow-ups from conversations.

This assignment is designed to evaluate backend engineering fundamentals, system design decisions, code quality, AI integration, and production-minded thinking.

We are not looking for the most complex implementation. We value clean architecture, maintainable code, robust error handling, thoughtful technical decisions, and reliable AI-powered workflows.

Estimated Effort

Estimated Time: 6–10 hours

Problem Statement

Build a **Meeting Intelligence Service** that helps users manage meetings, extract actionable insights, and stay on top of follow-ups.

The service should:

- Store meeting information and transcripts
 - Generate AI-powered meeting insights
 - Extract action items from transcripts
 - Provide citations for all generated insights
 - Track action item status
 - Detect overdue action items
 - Send reminders through a real third-party integration
-

Functional Requirements

1. Authentication

Implement authentication for protected APIs.

You may choose any reasonable authentication mechanism.

Examples:

- JWT Authentication
- Session-Based Authentication

Document your choice and rationale.

2. Meeting Management

Create Meeting

POST /api/meetings

Example Request:

```
{
  "title": "Sprint Planning",
  "participants": [
    "alice@example.com",
    "bob@example.com"
  ],
  "meetingDate": "2026-05-20T10:00:00Z",
  "transcript": [
    {
      "timestamp": "00:10",
      "speaker": "John",
      "text": "We should launch next Friday."
    },
    {
      "timestamp": "00:20",
      "speaker": "Alice",
      "text": "I will prepare release notes."
    }
  ]
}
```

Get Meeting

GET /api/meetings/:id

List Meetings

GET /api/meetings

Requirements:

- Pagination support
 - Filtering support (optional)
-

3. AI Meeting Analysis

Analyze Meeting

POST /api/meetings/:id/analyze

Use any LLM provider.

Examples:

- Gemini
- Groq
- OpenAI
- Claude
- OpenRouter
- Any equivalent provider

The endpoint should generate:

- Meeting Summary
- Action Items
- Decisions
- Follow-up Suggestions

Example Response:

```
{
  "summary": [
    {
      "text": "Team plans to launch next Friday.",
      "citations": [
        {
          "timestamp": "00:10"
        }
      ]
    }
  ],
  "actionItems": [
    {
      "task": "Prepare release notes",
      "assignee": "Alice",
      "citations": [
        {
          "timestamp": "00:20"
        }
      ]
    }
  ]
}
```

4. Grounding & Citation Requirement (Important)

All AI-generated content must be grounded in the provided transcript.

The system must not:

- Invent attendees

- Invent action items
- Invent decisions
- Invent meeting outcomes
- Add information not explicitly present in the transcript

Every generated insight must include at least one citation referencing the transcript segment(s) from which the insight was derived.

Examples of generated content that require citations:

- Summaries
- Decisions
- Action Items
- Follow-up Suggestions

Submissions generating unsupported or hallucinated information may lose evaluation points.

5. Action Item Management

Create Action Item

```
POST /api/action-items
```

Update Status

```
PATCH /api/action-items/:id/status
```

Supported statuses:

```
PENDING  
IN_PROGRESS  
COMPLETED
```

Get Action Items

```
GET /api/action-items
```

Support filtering by:

- Status
 - Assignee
 - Meeting ID
-

6. Overdue Detection

An action item is considered overdue when:

```
status != COMPLETED
AND
dueDate < current time
```

Get Overdue Action Items

```
GET /api/action-items/overdue
```

7. Scheduled Reminder Job

Implement a scheduled/background process.

The scheduler should:

1. Identify overdue action items
2. Trigger reminder notifications
3. Record reminder history

You may use:

- node-cron
 - Scheduled Jobs
 - Any equivalent mechanism
-

8. External Integration (Mandatory)

Implement one real third-party integration.

Choose one:

- Slack Webhook
- Discord Webhook
- Telegram Bot API
- Notion API
- Google Calendar API
- Email Provider (Resend, SendGrid, etc.)
- Any publicly documented API

The integration must be actively used by the reminder workflow.

Example reminder:

```
Reminder: Prepare release notes
Assigned To: Alice
Due Date: 2026-05-25
```

Simply configuring an SDK without using it in the workflow does not satisfy this requirement.

Non-Functional Requirements

Unified API Response Format

All APIs must return a consistent response structure.

Success Response

```
{
  "traceId": "abc123",
  "success": true,
  "data": {}
}
```

Error Response

```
{
  "traceId": "abc123",
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Meeting title is required"
  }
}
```

Request Traceability

Every request must include a trace identifier.

Requirements:

- Generate trace ID if absent
- Include trace ID in logs
- Include trace ID in all responses

Structured Logging

Logs should include:

- Timestamp
- Trace ID
- Request Method
- Request Path
- Response Status

- Error Details (when applicable)
-

Validation

Input validation is required.

Examples:

- Invalid email addresses
- Missing required fields
- Invalid status values
- Invalid dates
- Malformed requests

Meaningful validation errors should be returned to clients.

Error Handling

Implement centralized/global error handling.

The application should fail gracefully and should not crash because of invalid client input.

Database Design

You may use:

- PostgreSQL
- MySQL
- MongoDB
- SQLite
- Any reasonable database

Your schema and design choices must be documented.

API Documentation

Provide OpenAPI / Swagger documentation.

The API documentation should be publicly accessible.

Health Endpoint

Implement:

```
GET /health
```

Response:

```
{
  "status": "UP"
}
```

Deployment Requirements

Deploy the application publicly.

Requirements:

- Publicly accessible URL
- CORS enabled (*)
- APIs accessible for evaluation

Suggested platforms:

- Render
 - Railway
 - Fly.io
 - Vercel
 - Any equivalent platform
-

Evaluation Endpoint

Implement:

```
GET /api/evaluation
```

Response:

```
{
  "candidateName": "John Doe",
  "email": "john@example.com",
  "repositoryUrl": "https://github.com/example/project",
  "deployedUrl": "https://example.com",
  "externalIntegration": "Telegram Bot API",
  "features": [
    "Authentication",
    "AI Analysis",
    "Reminder Scheduler"
  ]
}
```

Submission Requirements

Submit the following:

1. Public GitHub Repository

Repository containing source code.

2. Live Deployment URL

Publicly accessible deployment.

3. Swagger/OpenAPI URL

Public API documentation.

4. README.md

Must include:

- Setup instructions
- Environment variables
- Local execution steps
- Deployment instructions
- API usage examples

5. DECISIONS.md

Document important technical decisions.

Examples:

- Database choice
- Authentication strategy
- External integration selection
- Project structure decisions

For each decision, explain:

- Why it was chosen
- Alternatives considered
- Trade-offs

6. AI_APPROACH.md

Explain:

- Prompt design

- Citation strategy
- Hallucination prevention approach
- Output validation strategy
- Known limitations

7. TESTING.md

Document:

- Test scenarios executed
- Edge cases considered
- Limitations discovered

8. CHANGELOG.md

Document implementation milestones and major changes.

9. CHECKLIST.md

Complete the checklist below and include it in the repository.

Submission Checklist

Mark completed items with [x].

Core Requirements

```
[ ] Public GitHub repository submitted
[ ] Application deployed and accessible publicly
[ ] README contains setup and run instructions

[ ] Authentication implemented
[ ] Database models designed and documented
[ ] Global error handling implemented
[ ] Unified API response format implemented
[ ] Request trace ID implemented and included in logs

[ ] Meeting analysis endpoint implemented
[ ] AI-generated insights include transcript citations
[ ] Hallucination prevention / grounding strategy implemented

[ ] Action item management implemented
[ ] Overdue action item detection implemented
[ ] Scheduled reminder job implemented

[ ] One real third-party integration implemented
[ ] Reminder notifications delivered through integration

[ ] Unit tests implemented
[ ] Input validation implemented
```

Bonus Milestones (Optional)

```
[ ] Docker support
[ ] CI/CD pipeline
[ ] Redis caching
[ ] Rate limiting
[ ] Integration tests
```

Evaluation Criteria

Category	Weight
Functional Correctness	25%
Code Quality & Maintainability	20%
AI Integration & Citation Accuracy	15%
Database Design	10%
Validation & Error Handling	10%
External Integration Quality	10%
Testing Quality	5%
Documentation & Technical Decisions	5%

What We Look For

A strong submission demonstrates:

- Clean and maintainable code
- Thoughtful technical decisions
- Well-designed APIs
- Reliable error handling
- Good database modeling
- Meaningful test coverage
- Grounded AI outputs with citations
- Clear documentation
- Production-minded engineering practices

A simple and well-engineered solution is preferred over an overly complex implementation.

Good luck, and we look forward to reviewing your submission.

— Team Hintro

